

An Introduction to Linux

The purpose of this tutorial/workshop is to provide an introduction and brief overview of the Linux operating system.

In the most general terms, there are two types of User Interfaces for Linux:

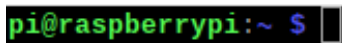
- CLI (Command Line Interface) also known as Text mode or Console mode. As the name implies, this is a method of working which involves only plain text input and output, generally happening either on the computer console, or in a "terminal emulator" window running on some other computer (such as your laptop).
- GUI (Graphical User Interface) This is similar to working on Windows and MacOS systems, with graphic displays, menus and application windows.

For this introduction we will be dealing only with using the CLI. This may seem to be a strange decision, since graphical desktops are common and easily available on Linux (just as they are on Windows and MacOS), but there are several good reasons for this decision.

- GUI desktops and graphical utilities often do some (or a lot) of the “dirty work” for you. That is nice for an ordinary user, but as a computer professional you should know how to install, configure and maintain a system yourself.
- During your career as a computer professional, you are very likely to have to access some server computers (such as routers, firewalls, web servers and such) via remote login, using telnet, ssh, or some other such text-oriented utility. In this case you will not have a GUI available, so you have to be able to work with a text-only interface.
- Servers installed in data centers (again, such as web servers, routers, firewalls, ids/ips systems and many others) are often installed “headless”, without any graphical display. Again, in these cases you are likely to have to use a text-only interface.

The Linux Shell (Command Interpreter)

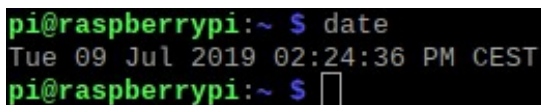
When you log into a Linux text console or terminal window, you will be presented with something that looks like this:

A screenshot of a terminal window with a black background. The prompt 'pi@raspberrypi:~ \$' is displayed in green and blue text, followed by a white cursor block.

```
pi@raspberrypi:~ $
```

The "\$" is the prompt from the Linux "command interpreter", also known as the "shell", which is the program that is ready to read your commands and execute them; this "prompt character" conveys two pieces of information: first, it tells you that the command interpreter is ready for input, and second it tells you that you are logged in as an "ordinary user", not an "administrator" (more explanation of this later).

At this point you can type any valid Linux command or application name, and you will see the result of that command. For example, to display the system date and time:

A screenshot of a terminal window with a black background. The prompt 'pi@raspberrypi:~ \$' is in green and blue. The command 'date' has been entered, and the output 'Tue 09 Jul 2019 02:24:36 PM CEST' is shown in white text. A new prompt 'pi@raspberrypi:~ \$' is visible below the output, followed by a white cursor block.

```
pi@raspberrypi:~ $ date
Tue 09 Jul 2019 02:24:36 PM CEST
pi@raspberrypi:~ $
```

You can try these commands yourself, and you should get (approximately) the same result. When there are differences in the result, stop and think to see if you can explain the differences.

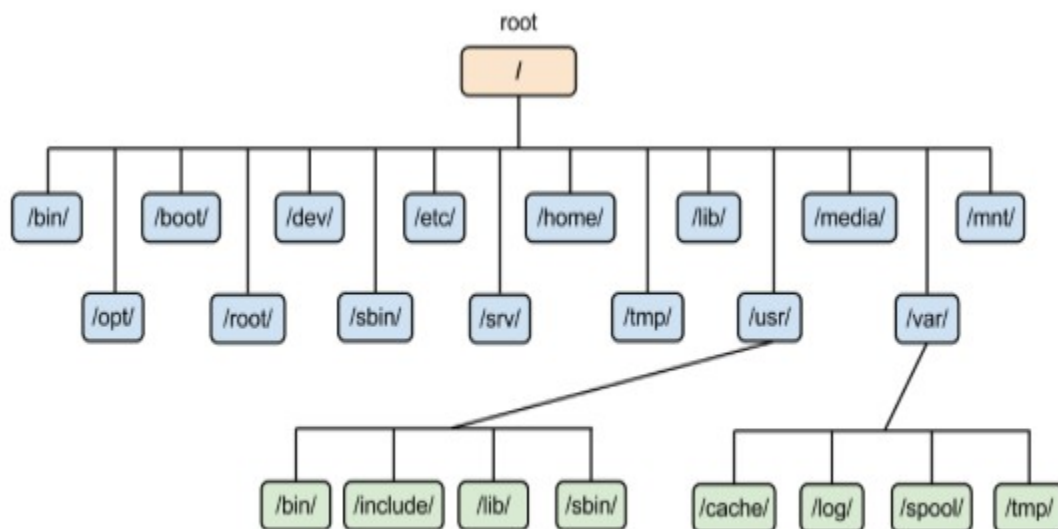
We will investigate and experiment with a variety of other Linux commands as we proceed through this workshop.

The Linux Filesystem

One of the most important concepts to learn about Linux is the structure and use of the filesystem. The filesystem is the way that Linux organizes the disk(s), and makes the information stored on them available to the user. When you work with Linux you simply must understand the filesystem, because it will be a crucial part of almost everything you do - running and configuring applications, reading public data and storing private data, and accessing all sorts of peripherals and external devices. If you don't understand the Linux filesystem, you are likely to feel "lost" in almost everything you try to do.

The Linux filesystem is organized in one hierarchical structure, no matter how many physical or logical disks are included in the system (there is no C: or D: drive). The entire hierarchy starts from a single directory (the Linux name for what Windows calls a "folder"), which is called the "root directory", and is represented by the symbol "/".

A directory on Linux can contain any number of files and other directories, and each of those directories can also contain additional files and directories. This results in a hierarchical structure which can be represented like this:



File system hierarchy of Linux

Any file or directory in the hierarchy can be uniquely identified by giving its "absolute pathname", which is the complete list of directories, starting from the root, which lead to that specific file or directory. Some examples that you can see in the diagram above are:

/

/etc

/usr/lib

Working with the Linux Filesystem

When you are using Linux, one of the most important concepts you must understand is your "present working directory" (also known as "current working directory"). This is the directory within the filesystem where you are "located", which means when you look for files, or create or delete files, this is the directory where those actions will take place. Note that in Linux, directories are represented by a forward slash "/" rather than the backslash "\" used in Windows. When you login to a Linux system, your working directory is automatically set to be your "home" directory. In most Linux systems, this is located in /home, with your login name. So for the generic pi account, the home directory would be

/home/pi

You can find out what your working directory is at any time with "pwd":

```
pi@raspberrypi:~ $ pwd
/home/pi
pi@raspberrypi:~ $
```

You can change your working directory with the "cd" command, and again, you can check to be sure the change of directory worked. Note also that you can see that the name of your present working directory is included in the shell prompt; when you are in your home directory it is abbreviated as "~", but when you are in any other directory it is shown as the complete pathname of the directory.

```
pi@raspberrypi:~ $ pwd
/home/pi
pi@raspberrypi:~ $ cd /tmp
pi@raspberrypi:/tmp $ pwd
/tmp
pi@raspberrypi:/tmp $
```

To get back to your home directory, you can simply use the "cd" command with no argument:

```
pi@raspberrypi:~ $ pwd
/home/pi
pi@raspberrypi:~ $ cd /tmp
pi@raspberrypi:/tmp $ pwd
/tmp
pi@raspberrypi:/tmp $ cd
pi@raspberrypi:~ $ pwd
/home/pi
pi@raspberrypi:~ $
```

Special Syntax in the Linux Filesystem

There are a few special names, symbols and abbreviations which are used in the Linux filesystem. We have already seen one of them, the “~” symbol represents your “home” directory, whatever that may be.

Two other very common abbreviations are “.” to represent your current working directory, and “..” to represent the directory one level up in the hierarchy from your current working directory. These abbreviations may seem a bit strange right now, but they are very useful, so we will see more details of how and when to use them later.

For those who are curious about more technical details, the implementation of these abbreviations might be interesting. The “~” is actually implemented in the command interpreter, not in the filesystem itself. That should make sense, because the filesystem doesn’t know what your home directory is, so when you type this symbol the command interpreter will look up that information and substitute your home directory for this symbol.

Conversely, the implementation of “.” and “..” is actually done in the filesystem, which also makes sense – the filesystem itself must always know what your current working directory is, and what the parent of that directory is. This brings up an interesting question – what is the value of “..” for the top-level (root) directory? There is obviously no “parent” for that highest directory! (Answer: in this special case, “..” is actually the same as “.”, the root directory)

Listing the Contents of a Directory

You can list the contents of a directory using the "ls" command. Without any argument, it will list the contents of your current working directory:

```
pi@raspberrypi:~ $ ls
Desktop  Downloads  Music      Public     Videos
Documents  MagPi     Pictures   Templates
pi@raspberrypi:~ $
```

What you see listed here are the default contents of the pi home directory. If you try this command on your system, you should see just about the same list. We will learn more about these soon.

You can give the "ls" command the name of a directory to list (this is called an argument), for example to list the contents of the root directory:

```
pi@raspberrypi:~ $ ls /
bin  dev  home  lost+found  mnt  proc  run  srv  tmp  var
boot  etc  lib  media      opt  root /sbin  sys  usr
pi@raspberrypi:~ $
```

This can be done for any other directory in the Linux filesystem hierarchy; try it with things like "ls /etc", "ls /home" or "ls /lib/firmware" to see what you get. What happens if you give the name of a directory which does not exist, like "ls /no/such/directory"?

The above examples use only the simplest form of the "ls" command, so they list only the names of the directory contents. If you want to see more information about each item, you can use the "-l" argument to get a "long" directory list:

```
pi@raspberrypi:~ $ ls -l
total 36
drwxr-xr-x 2 pi pi 4096 Jul 10 02:42 Desktop
drwxr-xr-x 2 pi pi 4096 Jul 10 02:42 Documents
drwxr-xr-x 2 pi pi 4096 Jul 10 02:42 Downloads
drwxr-xr-x 2 pi pi 4096 Jul 10 02:15 MagPi
drwxr-xr-x 2 pi pi 4096 Jul 10 02:42 Music
drwxr-xr-x 2 pi pi 4096 Jul 10 02:42 Pictures
drwxr-xr-x 2 pi pi 4096 Jul 10 02:42 Public
drwxr-xr-x 2 pi pi 4096 Jul 10 02:42 Templates
drwxr-xr-x 2 pi pi 4096 Jul 10 02:42 Videos
pi@raspberrypi:~ $
```

In this list you can see whether each item is a file or directory, the Linux access permissions (we will talk more about that later), the owner and group of the file, the file size in bytes, and the date and time the file was last modified. That's a lot of useful information!

There are many, many more command line options for the "ls" command. We will learn how to find those options later, with the "man" command.

Creating and Deleting Directories

You can create a new directory, for example to use for a project that you are working on, by using the "mkdir" command (make sure you are in your home directory before trying this command):

```
pi@raspberrypi:~ $ ls
Desktop  Downloads  Music      Public     Videos
Documents MagPi      Pictures   Templates
pi@raspberrypi:~ $ mkdir MyProject
pi@raspberrypi:~ $ ls
Desktop  Downloads  Music      Pictures   Templates
Documents MagPi      MyProject  Public     Videos
pi@raspberrypi:~ $
```

This can be repeated as often as you need to create other directories, and the "mkdir" command can be given more than one directory name to create:

```
pi@raspberrypi:~ $ cd
pi@raspberrypi:~ $ mkdir MyProject
pi@raspberrypi:~ $ ls
Desktop  Downloads  Music      Pictures   Templates
Documents MagPi      MyProject  Public     Videos
pi@raspberrypi:~ $ mkdir NewDirectory
pi@raspberrypi:~ $ mkdir DirectoryOne DirTwo ProjectB
pi@raspberrypi:~ $ ls
Desktop  Documents  Music      Pictures   Templates
DirectoryOne Downloads  MyProject  ProjectB   Videos
DirTwo   MagPi      NewDirectory Public
pi@raspberrypi:~ $
```

You can also create a directory hierarchy, to help you keep things separated:

```
pi@raspberrypi:~ $ mkdir MyProject/src MyProject/documents MyProject/bin
pi@raspberrypi:~ $ ls
Desktop  Documents  Music      Pictures   Templates
DirectoryOne Downloads  MyProject  ProjectB   Videos
DirTwo   MagPi      NewDirectory Public
pi@raspberrypi:~ $ ls MyProject/
bin documents src
pi@raspberrypi:~ $
```

Finally, you can remove directories with the "rmdir" command:

```
pi@raspberrypi:~ $ rmdir MyProject/documents/  
pi@raspberrypi:~ $ ls MyProject/  
bin  src  
pi@raspberrypi:~ $ rmdir MyProject/  
rmdir: failed to remove 'MyProject/': Directory not empty  
pi@raspberrypi:~ $
```

Fortunately, as you can see above, the rmdir command will not remove a directory which is not empty.

There are other ways, and other commands, to remove directories, including directories which are not empty. We will learn about those later as well.

Getting help with Linux Commands

We have already seen several Linux commands, but we have only looked at the simplest ways to use them. If you want more complete information about a Linux command, you can use the "man" command. For example, "man ls" produces the following output:

```
LS(1)                                User Commands                                LS(1)

NAME
    ls - list directory contents

SYNOPSIS
    ls [OPTION]... [FILE]...

DESCRIPTION
    List information about the FILES (the current directory by default).
    Sort entries alphabetically if none of -cftuvSUX nor --sort is speci-
    fied.

    Mandatory arguments to long options are mandatory for short options
    too.

    -a, --all
        do not ignore entries starting with .

    -A, --almost-all
        do not list implied . and ..

    --author
Manual page ls(1) line 1 (press h for help or q to quit)
```

The manual information is presented one screen at a time (we will learn about the utility which does this later). You can press the space bar to show the next screen, or "b" to go back to the previous screen, and "q" to quit when you are finished reading the man page.

On the man page you will find the description of the command, and a list of command line arguments (options) with a description of what they do. For example, "man ls" will show you that the ls command has an argument "-R" which lists directories recursively, as shown here:

```
pi@raspberrypi:~ $ ls -R
.:
Desktop      Documents    Music         Pictures      Templates
DirectoryOne Downloads     MyProject     ProjectB      Videos
DirTwo       MagPi        NewDirectory  Public

./Desktop:

./DirectoryOne:

./DirTwo:

./Documents:

./Downloads:

./MagPi:
MagPi82.pdf

./Music:

./MyProject:
bin  src

./MyProject/bin:

./MyProject/src:

./NewDirectory:

./Pictures:

./ProjectB:

./Public:

./Templates:

./Videos:
pi@raspberrypi:~ $
```

Look at the man page for "ls", and try out some of the other arguments to see what kind of output they produce.

Creating Files

You can create an empty file on Linux using the "touch" command:

```
pi@raspberrypi:~ $ touch MyFile
pi@raspberrypi:~ $ ls
Desktop    Documents  Music      NewDirectory  Public
DirectoryOne Downloads  MyFile     Pictures      Templates
DirTwo     MagPi      MyProject  ProjectB      Videos
pi@raspberrypi:~ $
```

You can not have two files with the same name in the same directory (duh), but you can have files with the same name in different directories:

```
pi@raspberrypi:~ $ touch MyFile
pi@raspberrypi:~ $ ls
Desktop    Documents  Music      NewDirectory  Public
DirectoryOne Downloads  MyFile     Pictures      Templates
DirTwo     MagPi      MyProject  ProjectB      Videos
pi@raspberrypi:~ $ ls DirTwo
pi@raspberrypi:~ $ touch DirTwo/MyFile
pi@raspberrypi:~ $ ls DirTwo/
MyFile
pi@raspberrypi:~ $
```

This brings up an important concept in the Linux filesystem, which was mentioned briefly in the introduction of the Linux filesystem above – every file is uniquely identified only by specifying its absolute pathname, that means the complete path from the root directory (“/”) to that file. In the case of the two files which we just created, the first one is located in your home directory, so its absolute pathname would be

/home/pi/MyFile

The second one is located in a subdirectory, so its absolute pathname would be

/home/pi/DirTwo/MyFile

Files can also be identified using a “relative pathname”, but as the name implies, this identification is then relative to the point from which you are looking (or specifying) the file. Again, in the example above, starting from your home directory the relative pathname of the first file is simply MyFile, while the relative pathname of the second file is DirTwo/MyFile. But if your current working directory were anywhere else in the Linux filesystem than your home directory, then trying to access those files using these relative pathnames would not work.

Deleting Files

You can remove a file with the "rm" command:

```
pi@raspberrypi:~ $ ls
Desktop      Documents  Music      NewDirectory  Public
DirectoryOne Downloads  MyFile     Pictures      Templates
DirTwo       MagPi     MyProject  ProjectB      Videos
pi@raspberrypi:~ $ rm MyFile
pi@raspberrypi:~ $ ls
Desktop      Documents  Music      Pictures      Templates
DirectoryOne Downloads  MyProject  ProjectB      Videos
DirTwo       MagPi     NewDirectory  Public
pi@raspberrypi:~ $
```

The “rm” command only works by default on files, it will not remove a directory. However, there is a command line argument for “rm” which tells it to also remove directories. Can you find out what that argument is? Where would you look to find this information?

Display the Contents of a File

To see the contents of a (text) file, you can use the “cat” command (the name is taken from the English word “concatenate”). For example, there is a “Message of the Day” file included on most Linux systems, called `/etc/motd`, so to read the contents of that file you could do this:

```
pi@raspberrypi:~ $ cat /etc/motd

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
pi@raspberrypi:~ $
```

This works well if the content of the file is less than the size of your screen. But if the file is longer than that, the “cat” command will simply scroll everything off the top of the screen until it reaches the end of the file, which is not very useful if you want to see the entire file (unless you can read *very* fast).

There are two Linux commands which avoid that problem, called “more” and “less”, both of which present the contents of a file one screen full at a time – very similar to the way the “man” command displayed manual pages. For example, if this tutorial were stored in a text file in your home directory:

```
pi@raspberrypi:~ $ more LinuxIntro.txt
An Introduction to Raspbian Linux

The purpose of this tutorial/workshop is to provide an introduction and brief overview of the Linux operating system, using Raspbian GNU/Linux as an example.

In the most general terms, there are two types of User Interfaces for Linux:

- CLI (Command Line Interface) also known as Text mode or Console mode. As the name implies, this is a method of working which involves only plain text input and output, generally happening either on the computer console, or an a "terminal emulator" window.

- GUI (Graphical User Interface) This is similar to working on Windows and MacOS systems, with graphic displays, menus and application windows.

For this introduction we will be dealing only with using the CLI.

Expected Hardware and Software

For this tutorial/workshop, it is expected that you will be working with a Raspberry Pi (any model), and a reasonably current version of the Raspbian GNU/Linux operating system. Raspbian should be configured to boot to the text console, no
--More-- (5%)
```

At this point you can press the space bar to show the next screen, or “b” to go back to the previous screen, or press “h” (for “help”) to see a list of other possible commands. To exit this utility, you simply enter “q”.

File and Directory Operations and Manipulation

Within the Linux filesystem, you can have any number or combination of files and directories. There are various Linux utilities for managing and manipulating those.

For example, the "mv" command (the name is derived from "move") can be used to change the name of a file or directory, or to move a file or directory to another location. Suppose that you had saved a text file with the name "file", and forgot to give the filename extension. You could use the command shown below to rename the file.

```
pi@raspberrypi:~ $ ls
Desktop    Documents  LinuxIntro.txt  MyProject      ProjectB  Videos
DirectoryOne Downloads  MagPi          NewDirectory   Public
DirTwo     file       Music          Pictures       Templates
pi@raspberrypi:~ $ mv file file.txt
pi@raspberrypi:~ $ ls
Desktop    Documents  LinuxIntro.txt  MyProject      ProjectB  Videos
DirectoryOne Downloads  MagPi          NewDirectory   Public
DirTwo     file.txt   Music          Pictures       Templates
```

You can also use the "mv" command to move a file to a different directory. As a simple example, the file which we just renamed is (obviously) in our present working directory. We could move that file to the "Documents" directory with the simple "mv" command shown here:

```
pi@raspberrypi:~ $ ls
Desktop    Documents  LinuxIntro.txt  MyProject      ProjectB  Videos
DirectoryOne Downloads  MagPi          NewDirectory   Public
DirTwo     file       Music          Pictures       Templates
pi@raspberrypi:~ $ mv file file.txt
pi@raspberrypi:~ $ ls
Desktop    Documents  LinuxIntro.txt  MyProject      ProjectB  Videos
DirectoryOne Downloads  MagPi          NewDirectory   Public
DirTwo     file.txt   Music          Pictures       Templates
pi@raspberrypi:~ $ mv file.txt Documents
pi@raspberrypi:~ $ ls
Desktop    Documents  MagPi          NewDirectory   Public
DirectoryOne Downloads  Music          Pictures       Templates
DirTwo     LinuxIntro.txt  MyProject      ProjectB       Videos
pi@raspberrypi:~ $ ls Documents
file.txt
```

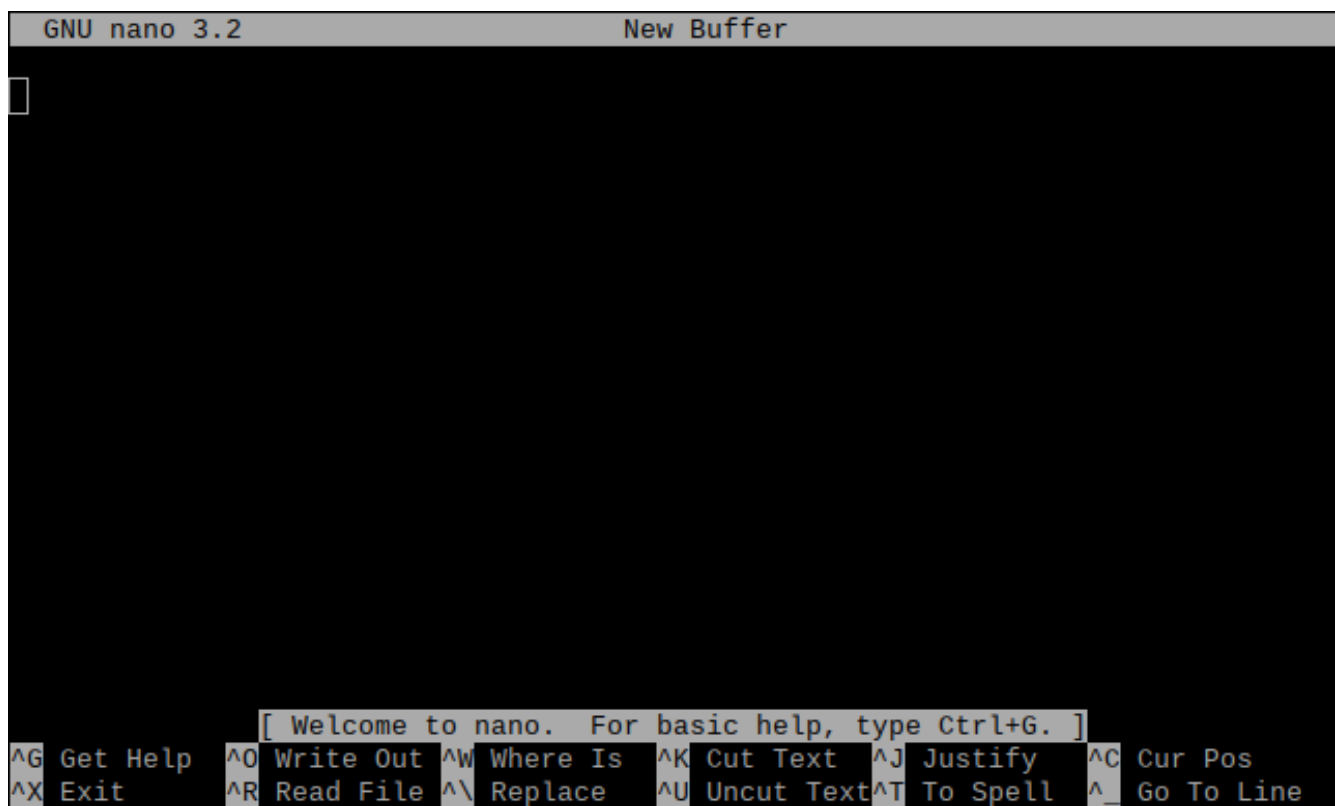
As you can see in this screen shot, if you give only the directory name for the destination when moving a file, the file will keep its original name in the new location. But these two commands could have been merged into one by giving both the destination directory name and the new filename: "mv file Documents/file.txt". You can try this on your own machine and see how simple, powerful and flexible the "mv" command is.

The "mv" command also works the same way on directories, for both renaming and moving. It should be obvious that when you "move" a directory, anything that was contained in the directory goes with it.

Creating and Editing Text Files

So far we have only created empty files, by using the Linux "touch" command. It is obviously much more interesting and useful to create and edit files containing some kind of text. One way to do that is to use a text editor; there are a lot of different text editors available on Linux, but we are going to concentrate on "nano", which is a very simple text-mode (not GUI) editor. (Another popular alternative is "vi" or "vim", which is much more powerful but also much more complex to use. If you are interested in vi, just ask one of the teachers for help, or look online for a “Getting Started with Vi” document).

You can start nano by simply typing its name at the shell prompt. You may also type the name of the file you want to edit at that time, but it is not absolutely required:



```
GNU nano 3.2          New Buffer

[ Welcome to nano.  For basic help, type Ctrl+G. ]
^G Get Help  ^O Write Out ^W Where Is  ^K Cut Text  ^J Justify  ^C Cur Pos
^X Exit      ^R Read File ^\ Replace  ^U Uncut Text ^T To Spell ^_ Go To Line
```

This will take you into a screen with the file name at the top (or "New Buffer" if you didn't give a filename), and some hints about commands and control characters at the bottom. You can then simply type whatever text you want in the file.

When you are finished editing the file, you can enter ^O (control-O) to save the file to disk. If you started nano without giving a file name, you will have to enter the name at this point. After you have saved the file, you can exit nano by entering ^X (control-X). If you try to exit nano without saving the changes you have made, it will prompt you to confirm that you want to discard the changes.

The file will be saved in your current working directory (unless you enter a directory name as part of the filename). You can then check that the file was created/saved by using the "ls" command that was mentioned above. Assuming that you used the name "file.txt", it would look like this:

```
pi@raspberrypi:~ $ ls -l file.txt
-rw-r--r-- 1 pi pi 15 Aug 23 09:49 file.txt
pi@raspberrypi:~ $
```

We will discuss the details of this line later; what is important right now is that the file exists (because it is listed), and it has a size of 14 characters.

You can then edit that file again by giving the filename along with the nano command:

```
$ nano file.txt
```

Students are strongly encouraged to learn at least basic editing with nano (or vi), because a very large portion of what you will have to do when working with Linux involves text editing. Help is always available within nano by entering ^G (control-G).

Summary

So far, we have covered the following important Linux concepts and commands:

- The Linux Shell
- The Linux Filesystem
- Your Home Directory
- File and Directory Operations
- Creating and Editing Text Files
- Linux commands
 - pwd
 - cd
 - ls (and ls -R)
 - mkdir
 - rmdir
 - man
 - touch
 - rm
 - cat, more and less for displaying files
 - mv to rename or relocate either files or directories
 - nano (or vi) for editing text files